The Grassroots Super-App and App Store

Anonymous Author(s)

Abstract

Grassroots platforms are owned, operated and governed by their participants. Each is specified by a grassroots contract, a few clauses in plain English stating who may do what and with whose consent, and is compiled from the contract into a program that its participants run on their smartphones, with no operator in the middle. A smartphone app today is distributed by an app store and run by an operating system, both controlled by others; existing super-apps, WeChat and Alipay, are controlled by an operator that admits and withdraws their mini-apps.

Here, we present the Grassroots App, a super-app, and the Grassroots App Store. The Grassroots App is one app on a person's smartphone that runs the grassroots social graph and, as mini-apps, grassroots programs compiled from grassroots contracts, all on one virtual machine for Grassroots Logic Programs (GLP) over a networking layer that connects smartphones by proximity, by the local network and over the Internet. It provides (*i*) friendships, established by physical encounter, by legacy online connection or by introduction; (*ii*) recovery of identity and state by friends serving as custodians; (*iii*) friend-to-friend distribution of grassroots programs, each with an invitation to join under its contract; (*iv*) the activation of certified grassroots programs and their isolation; and (*v*) contact between two participants of a program who are not friends.

A developer of a grassroots program is a person with a key. A program is the compiled code, signed by its compiler, which gives the contract it is compiled from; its developer publishes it by signing it, attesting that it is the compilation of that contract and behaves according to it, and a Grassroots App receiving it checks the contract itself. A person trusts the developers they choose, as they trust compilers: the Grassroots App holds the keys of the developers its person trusts, and a person adds a developer by their key or upon a friend's invitation to that developer's program. A developer publishes a program to friends, who pass it on friend to friend with the developer's certification. The Grassroots App Store is one such developer, whose site holds the grassroots contracts of the GLP repository, the artefacts the cascade produces from each contract down to its compiled program, and a live demonstration of each; anyone can run a store.

The security of the Grassroots App rests on the correctness of the virtual machine, of the social graph and of the compiler, and on the smartphone attesting the application it runs. Given these, a grassroots program reaches only what the Grassroots App granted it, cannot exhaust the smartphone's resources, and its traffic passes between attested Grassroots Apps running its certified code. The social graph, social networking, currencies and the other platforms of the store run as grassroots programs of the prototype.

CCS Concepts: • **Software and its engineering** → **Operating systems**; **Concurrent programming languages**; • **Security and privacy** → *Trusted computing*; • **Computer systems organization** → *Peer-to-peer architectures.*

Keywords: Grassroots Platforms, Super-App, Mini-Apps, Operating Systems, Concurrent Logic Programming, Virtual Machine, Isolation, Remote Attestation, Social Graph

1 Introduction

The majority of humanity uses smartphones to access global platforms owned, operated, and controlled by others, centralised/autocratic (Facebook) or decentralised/plutocratic (Bitcoin) [53, 74], with those in control extracting unprecedented economic, political, and social value from their users.

Grassroots platforms [50, 52] are designed to be owned, operated and governed by their participants. They are person-to-person smartphone apps that realise Proudhon's two-centuries-old concept [42, Sixth Study]: a social contract that can have multiple, independent, volitionally-forming instances — (*i*) one could "make a contract with all, as … with some" — that may coalesce by mutual consent, possibly into a single global instance — (*ii*) "each group of citizens … formed by a like contract … could thereafter, and always by a similar contract, agree with every and all other groups". A grassroots platform is constituted from its participants' smartphones and the grassroots app each participant runs. We hope that grassroots platforms owned, operated and governed by their participants will become the foundation of an egalitarian and democratic digital realm [52].

A grassroots platform is specified by a *grassroots contract* [20]: a few clauses in plain English, each stating who may do what and with whose consent, with its pragmas: the interface pragmas that name the app's tabs, buttons and questions, and the implementation pragmas that say what the app does beyond its screens. The contract is rendered as speech-act schemas, checked to be grassroots by three decidable conditions, and compiled down an *abstraction cascade* [20, 28, 34] into a program in Grassroots Logic Programs (GLP) [54, 59, 60] that its participants run on their smartphones. Contracts so compiled include keys and knowledge, the social graph and social networking [20, 51], child-safe social networking [55], social networking with AI agents [56], coins and bonds [49, 57], and feeds and followers, barter and ride-hailing (in preparation); grassroots democratic federations [23, 27, 63] have been specified.

Super-App	Existing (WeChat, Alipay)	Grassroots
Host	Operator's servers with users' smartphones	The person's smartphone
Control	Operators (S5, S8) [2]	Person
Mini-app	A package of markup, style and script, run on a JavaScript thread and WebView threads, with a storage space of its own (S2) [67]	A grassroots program: the GLP program compiled from a grassroots contract down the abstraction cascade; all run concurrently on the GLP VM
Contract	End-User License Agreement, undertaken by the user to the operator	Grassroots contract, one for the Grassroots App and one per program, undertaken by each participant to all others
Store	The operator's, which admits a mini-app after review and withdraws it at will (S5, S12)	Any developer may run one; the Grassroots App Store is one (Section 7)
Code Admission	Developer registration and code review (S12)	Certification by the developer and by the compiler; trust in a developer is the person's; an invitation by a friend (Section 5)
Capabilities	Operator's services — payments, location, camera, login — each granted by permission (S1, S3, S10)	GLP channels to the Social Graph and the UI; the mini-app has a GLP channel to the friend's mini-app and via it to other participants
Identity	Account the operator issues, with per-mini-app identifier (S11, S13)	Self-chosen key pair
Recovery	Of the account, by the operator	Of the person's identity, and of each mini-app's state, by friends serving as custodians (Section 4.3)
Inter-app communication	Allowed channel the client passes (S4)	Mini-apps communicate with the Grassroots Social Graph; inter-app communication (Section 8.2) is a future extension
Who vouches	Operator	A participant running the same mini-app (Section 8)

Table 1. Existing vs. Grassroots Super-Apps. A *GLP channel* consists of two streams in opposing directions; each channel end consists of a reader of one stream and a writer to the other. S1–S13 are the security mechanisms of the super-app in the SoK of Yang et al. [69]; Appendix B presents all thirteen and ten threats.

VM	Erlang (BEAM)	GLP
Process	Has own heap and stack; 327 words when spawned [16]	A record with program counter and pointers into the heap, a few words (subject to partial-evaluation of meta-interpreter) [59]
Memory	Heap and stack per process; process-to-process messages are copied [16]	Heap shared by the social graph and all mini-apps; local messages shared on the heap
Cross-agent message	A ground copy of a term in a mailbox; reply to the sender's process identifier	Single assignment to a writer, sent to its paired reader; message may carry logic variables (readers/writers), each an input/output channel to any agent holding their other end [54]
Distribution	A node, `name@host`, located through a port mapper daemon [17]	A person's smartphone, named by the person's public key, accessible to friends, via physical proximity (BLE, LAN), legacy online connections, or a rendezvous server
Trust	One cookie per cluster; connections transitive and fully connected by default; one administrative domain [17]	Between friends, who know each other directly or through a mutual friend; contact through a mini-app opens with mutual attestation and exchange of undertakings (Section 8)
Isolation	The address space: a heap per process	Mini-app holds only what the super-app grants it (Section 8.2)
Resource bounds	Preemption by reduction count; no reduction budget per process	The VM schedules; mini-app execution controlled and resource-bounded by meta-interpreter (to be partially-evaluated into a transformer) (Section 8.3)
Code	Hot-code loading of named modules [17]	Certified program received from friend and activated [60]
Types	Dynamic	Moded and polymorphic, checked by the compiler [60]; dynamic extension will be needed for inter-app communication

Table 2. Erlang VM vs. GLP VM.

A smartphone app today is distributed by an app store, which admits it after review and may withdraw it, and run by an operating system; both are controlled by others. A *super-app* is a smartphone app that can run many applications, referred to as *mini-apps*, with WeChat and Alipay as prominent examples [2, 3]. The security literature classifies super-apps as operating-system-like: a super-app supplies isolation, authentication, permissions, application distribution, and lifecycle management [69]. Existing super-apps are controlled by an external operator that decides which mini-app may be installed, grants and revokes their capabilities, holds the accounts through which people are identified, can remove a mini-app from every phone at once, and binds each person by an End-User Licence Agreement [30, 43].

In this paper, we present the *Grassroots App*, a super-app, and the *Grassroots App Store*. The Grassroots App is one app on a person's smartphone that runs the grassroots social graph and, as mini-apps, *grassroots programs* compiled from grassroots contracts. We write Grassroots App and grassroots program, and keep super-app and mini-app where we compare with WeChat and Alipay (Table 1, Section 10). Unlike existing super-apps, the Grassroots App is not controlled by an external operator, but by the person operating it. It supports the attested person-to-person distribution and deployment of certified grassroots programs, whose participants are bound to each other by a grassroots contract: a mutual and symmetric contract among its participants, adopted by each of them [11, 20, 34]. To join a grassroots program a person signs an undertaking to keep its contract, presented to each participant upon first contact through the program [20]. Table 1 compares the two architectures.

The Grassroots App consists of a Virtual Machine (VM) for GLP [54, 59, 60], the Secure Grassroots Social Graph (SGSG) [15], and a networking layer that connects smartphones by proximity, over Bluetooth Low Energy and the local network, and over the Internet through rendezvous servers that anyone can run (in preparation). Together they provide the functions for the deployment and operation of grassroots platforms as programs of the Grassroots App: (*i*) establishing friendships via physical encounters or legacy online connections, both over the networking layer, and introductions via a mutual friend [51]; (*ii*) identity and state recovery via friends that serve as custodians [15]; (*iii*) friend-to-friend distribution of grassroots programs, each with an invitation to participate under its contract; (*iv*) the activation of certified grassroots programs, joining their instance and granting them access to the Grassroots App user interface; and (*v*) contact between two participants of a program who are not friends, by an introduction through a person who knows both or by each person's consent to knowing the other.

A *developer* of a grassroots program is a person with a key, who writes its contract. A program is the compiled code, signed by its compiler, which gives the contract it is compiled from; its developer publishes it by signing it, attesting that it is the compilation of that contract and behaves according to it, and a Grassroots App receiving it checks the contract itself [20]. A person trusts the developers they choose, as they trust compilers: the Grassroots App holds the keys of the developers its person trusts, and a person adds a developer by their key or upon a friend's invitation to that developer's program. A developer publishes a program to the people who know them, who pass it on friend to friend with the developer's certification. The Grassroots App Store is one such developer: its site, at https://peoplesweb.org, holds the grassroots contracts of the GLP repository, the artefacts the cascade produces from each contract down to its compiled program, and a live demonstration of each program; anyone can run a store (Section 7).

The VM runs the SGSG and the grassroots programs, the execution of each being many goals (featherweight processes) sharing a heap and logic variables (point-to-point single-message channels) shared across smartphones [54, 59]. Unlike the Erlang VM (BEAM) [6], the one widely deployed machine built for lightweight communicating processes, all processes share a heap, not a heap per process; a message may carry logic variables, not only a ground copy of a term; and the unit of distribution is a person's smartphone, not a node of a cluster. Table 2 compares the two virtual machines; Section 3 presents the GLP VM.

The security of the Grassroots App depends on the correctness of the virtual machine (implemented in Dart/Flutter [19, 21]), of the SGSG (implemented in GLP), of the GLP compilers that compile the SGSG and the grassroots programs [60], and on the smartphone attesting the application it runs [4]. Given these, we argue that a grassroots program reaches only what the Grassroots App granted it, so it cannot interfere with the Grassroots App or with other programs. Moreover, the trusted compiler maps any program into a resource-bounded program that cannot exhaust the smartphone's resources at the expense of the Grassroots App or other programs. This transformation is specified by a meta-interpreter that bounds its resources, reports its failures and can abort it [46] and realised by a transformation rationalised as its partial-evaluation wrt the compiled program [24, 26].

A grassroots program arrives with three certifications [20]: by its developer; by the compiler; and by the friend who invited one to join its instance. Whether a program is harmless hinges on its developer, with the inviting friend providing some comfort; the correctness of the compiler and of the Grassroots App guarantee that it cannot harm the Grassroots App or other programs.

The social graph, social networking [51] and grassroots coins [49], each compiled from its contract, run as programs of the Grassroots App prototype [28]; social networking, including a child-safe one [55], and grassroots currencies [49, 57] have run on a physical smartphone. The VM is implemented in Dart, with Flutter for the user interface; the SGSG

and the grassroots programs are GLP programs. The repository holding the VM, the SGSG, the grassroots programs and the store is at https://github.com/EShapiro2/GLP. The semantics of GLP, its type system, and the correctness of the VM against the semantics are prior work [54, 59, 60], as are the specification and the recovery protocols of the secure social graph [15], the networking layer beneath it (in preparation), the elicitation of volitions by the user interface [28], and grassroots contracts and their compilation [20]. Here, we present the Grassroots App, the services it provides a grassroots program, the store, and the programs running on the prototype.

Section 2 presents the Grassroots App; Section 3 the VM as a system; Section 4 the contracts built into the Grassroots App, physical encounters and the social graph; Section 5 the dissemination of programs; Section 6 the programs that arrive by it; Section 7 the Grassroots App Store; Section 8 what the security of the Grassroots App assumes and proves; Section 9 the prototype; Section 10 related work.

2 The Grassroots App

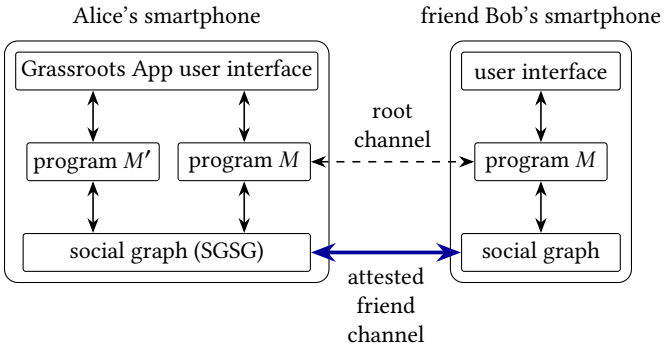


Figure 1. The Grassroots App on a smartphone: the social graph and the grassroots programs run on one GLP VM; a program holds two GLP channels, to the social graph and to the user interface, and none to another program; the social graph holds an attested channel to each friend, over which it opens, with a handshake, the root channel of a conversation between two executions of the same program.

A grassroots contract becomes a running platform through an *abstraction cascade* [20, 28], each step implementing the one above it; this paper takes the cascade from the GLP program downwards. An *agent* consists of a person, identified by the public key of a self-chosen key pair, and a smartphone operated by that person [34]. A grassroots program is the GLP program of one platform, which every participant of it runs on their own smartphone [11]; the task of the Grassroots App is to run multiple grassroots platforms, in the form of grassroots programs, concurrently.

The Grassroots App is one app on a person's smartphone. It consists of the GLP virtual machine (Section 3), the networking layer beneath it, which with every service below GLP is specified in a paper of its own, GNA (in preparation), and the contracts built into it, GKK, GPE and GSG — keys and knowledge, physical encounters and the social graph (Section 4) — which come with the Grassroots App and are disseminated to nobody: a person enters the social graph by installing the Grassroots App, and every other grassroots platform reaches them by dissemination (Section 5) and runs on it as a grassroots program. To the phone's operating system the Grassroots App is one application, and a grassroots program is a certified GLP program it runs (Figure 1).

What the Grassroots App grants a program. A grassroots program holds, at activation, the variables the Grassroots App posted with its goal, and afterwards only what it creates itself and what arrives on them. The Grassroots App grants a program two GLP channels: (*i*) a GLP channel to the social graph, over which the program asks the social graph to invite a friend, and the social graph hands the program, at each invitation (Section 5), one end of a root channel to the execution of the same program at another participant; and (*ii*) a GLP channel to the Grassroots App user interface, through which the program presents requests to its person and receives the person's responses [28]. On the two the Grassroots App assigns only ground terms, fresh variables whose other ends it holds, and, on the social-graph channel, the root-channel ends; this is its *grant discipline*. The Grassroots App grants a program no GLP channel to another program. The system predicates and kernels that reach the network or the person are: `send_to_net/1` and `send_to_user/1`, the five system predicates of the networking layer, `peer_address/2`, `peer_app/2`, `punch_udp/1`, `declare/1` and `remove/1` (GNA, in preparation), `authorise_link/2`, and the kernels beneath them. These are the *OS-privileged predicates*; a certified program calls none of them, the compiler refusing a certificate to a program that reaches one (Section 8.4), so a program sends only on the two channels. A program therefore reaches another person only through an execution of itself, and through it other participants, and reaches its own person only through the Grassroots App user interface. Local inter-program communication is a future extension. Access to any other hardware component of the smartphone will be mediated by the VM and provided via a dedicated GLP channel.

Storage and recovery. The Grassroots App gives each program private storage, sealed by the platform to the Grassroots App (Section 8); recovery is Section 4.3's topic.

3 The GLP Virtual Machine

The social graph and every program run on one machine: the multiagent, message-passing implementation of GLP, derived from the language's semantics and proved correct against it [59]. That paper defines the machine as a formal

transition system and proves it correct; this section describes it as a system, in the terms of Table 2.

One heap per person, featherweight processes, and variables as channels. A person's smartphone runs one GLP agent, named by the person's public key; its program is a resolvent of goals, a goal a record with a program counter and pointers into the agent's one heap, and a variable a pair of cells, writer and reader, the writer bound at most once; a goal suspended on a reader is activated when the writer is bound, so a suspended goal costs its record and no time [54, 59]. A variable shared across agents is two local pairs joined by a global link, the value sent to the reader's agent when the writer is assigned; the term sent may carry further readers and writers, each a link of its own, so a message brings the channel it is answered on, and a stream is an ongoing channel from one initial sharing. A *cold call* establishes a shared variable between two agents that have none; every other cross-agent communication is an assignment to a variable the two already share [59].

Compiled programs as values. A GLP program compiles to a value: the code and a certificate over its source and compiled identities (Section 5) [60]. `run(Goal, Type, Program)` activates the compiled program and posts `Goal` to it; a program activates only if it type-checks, and `run/3` errs unless `Type` is the type identity the program's exports record for the goal's predicate [54]. A program that activates programs it was not compiled with, as the Grassroots App does, holds no declaration of their types: the activated program declares its entry as the instance of the activator's entry template at its own types, and its exports record the identities of those types; the activator obtains them from the program value, and `Type` is the type identity of the template's *opaque instance* at them [60]. Every goal points to a compiled program, and `self_module` returns, as a constant, the compiled program that calls it. An agent ships a running program to a friend by sending this value over a channel; the friend verifies the certificate and runs the code. Types are moded and polymorphic, and are checked before a program is activated, so no term is inspected at run time [60].

Compared with the Erlang VM. Table 2 states where the two machines differ. They agree on more than either agrees with a thread-and-lock runtime — lightweight processes, no shared mutable state, asynchronous message passing, and code replaced while the system runs — and they differ on what a message carries and on what a unit of distribution is. A GLP channel the Grassroots App grants is a term it passes, and the program that receives it holds it in the same heap. On the BEAM the corresponding grant is a copy into another process's heap, and every later message is another copy. The cost of the shared heap is that isolation cannot come from the address space, which is the subject of Section 8.2.

4 The Built-in Contracts

Three contracts come with the Grassroots App, installed with it and disseminated to nobody: GKK, keys and knowledge, GSC's [20]; GPE, physical encounters; and GSG, the social graph. This section presents the last two and the program of the social graph.

Physical encounters. Two people near each other, on the same local network or within radio range, come to know each other with no infrastructure, by the contract GPE, which extends GKK; the text in the box is the repository's copy, `programs/social/graph/GPE.txt`.

GPE: Grassroots Physical Encounters

GPE extends GKK [20].
A person is *nearby* Alice when her phone reaches theirs by Bluetooth or the local network, without the Internet; a *place* is where Alice is. Alice and Bob are persons.

1. ■ *Open Place.* Alice may declare the place she is in open, and may close it.
2. ■ *Encounter.* Alice and Bob, nearby each other, who have both declared the place open, may know each other.

Interface pragmas

Nearby: the persons nearby.
Open Place: `Alice [Open place] [Close place]`

Implementation pragmas

Open Place: while the place is open Alice's app sets the trust of Bluetooth and of the local network open, and closed otherwise (GNA).
Nearby: an environment predicate, `nearby(Alice, Bob)`, the persons the networking layer reports nearby, which Alice's app keeps current [20].
Encounter: `Alice stands on [Open place] until [Close place]`; the two phones exchange keys (GNA, open trust).

The networking layer gives each agent a presence by which the agents that know it recognise it and nobody else identifies it, and answers an unrecognised presence on a proximity underlay only while its trust is open (GNA, in preparation); knowing a person is knowing their key, so two phones that exchanged keys under open trust have met, GPE:2, and the Grassroots App shows its person the persons nearby it knows, friends among them wherever they are. Two who know each other befriend by GSG:1 over the channel between them.

The *social graph* is the grassroots contract GSG, which extends GKK, whose three clauses are a person's identity by a key pair of their own, acquaintance by an act of both, and introduction by a person who knows both [20]. GSG adds friendship:

GSG: Grassroots Social Graph [20]

GSG extends GKK [20].
Alice and Bob are persons. Alice *trusts* Bob when she knows Bob personally and believes him to keep the undertakings he signs under grassroots contracts.

1. ◆ *Friendship.* Alice and Bob, who trust each other and are not friends, may jointly decide to become friends.
2. ■ *Ending a friendship.* Alice may end her friendship with Bob.
3. ○ *Loss of Trust.* Alice ends her friendship with Bob when she ceases to trust him.

Pragmas

Friends: the friends of Alice.
Friendship: `Alice [Befriend]`
`Bob: Alice asks to be your friend.`
Ending a friendship: `Alice [Unfriend]`

A friendship is mutual trust on record, so the social graph is a web of trust [72, 73]; once Alice knows that Bob breached an undertaking she can no longer trust him, and by GSG:3 she must end the friendship. The text in the box is the repository's copy, `programs/gsc/GSG.txt`, and the program of the social graph is compiled from it [20, 28]. The social graph is adopted at befriending, each party presenting to the other its one-sided *undertaking* to keep it towards all its participants [20]. Its state is personal, each agent's friend set, and pairwise, each edge's conversation, so no two participants need to agree on a shared total order [51]; it is proved grassroots [20, 34, 51, 52].

The social graph's secure version adds, for each agent, an identity record and a record of each friend's friends, and two transitions, Replace and Recover, that repair the loss of a key and the loss of a smartphone [15]; it is proved grassroots and to implement the plain one at quiescence [15, 34, 58].

4.1 The Program

The social graph's program is the platform the compiler prints from GPE and GSG together with GPD and GAS, one platform for the four contracts (Appendix A): its agent, one goal per person, reads the messages of its peers and its person's answers, and writes its messages, its view and its questions [28]. The Grassroots App is that platform with the operating system around it, the GLP program the runtime posts with the initial goal of every agent, the person channel and the network channel, and nothing else [54]: the operating system runs the platform's agent with its dispatcher on the person channel, and carries the agent's messages, to a friend over the friend channel and to anyone else by `send_to_net/1` (Section 2). When the stream of the person's acts closes the Grassroots App ends: the agent closes its friend channels and the channels of every program it hosts, whose peers see a crash (Section 8.1). Friends communicate over GLP channels (Table 1); an end of one is the term `ch(In, Out)`, and two agents hold the two ends of the same pair of streams:

```
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
receive(X?, ch([X|In], Out?), ch(In?, Out)).
```

A single call `new_channel(PQCh, QPCh)` binds the reader `Xs?` of one end to the writer `Xs` of the other. Befriending, GSG:1, establishes the *friend channel* that carries the two friends' subsequent messages; unfriending, GSG:2, sends `unfriend` on the friend channel and closes it at both ends (Appendix A).

4.2 Establishing Friendships

The three ways of establishing a friendship differ in how the two parties come to address one another; the befriend transaction and its two volitional guards are the same in all three [34]. Declining is not an act a person wills but the absence of the will to befriend, so it has no clause of its own; the agent answers the other party all the same, binding its consent variable to `no`, so that nobody waits on an answer that will not come.

Legacy online connections. Two people who are not near each other, and have no mutual friend in the social graph, become friends over the connections they already have. The social graph generates an *invitation link*, signed with `sign/3` and carrying the inviter's key, a rendezvous server the social graph declared, a serial and an expiry, registers the serial at that server, and hands the link to its person as a web address, https://peoplesweb.org/i followed by # and the link's text in the user interface's canonical encoding [28], to send over any legacy channel: email, a messaging app, a web page; tapped on a smartphone, the address opens the Grassroots App where it is installed, and otherwise a page that says how to get it (GNA, in preparation). The recipient's social graph verifies the inviter's signature and the expiry, declares the inviter a peer, redeems the link at the rendezvous agent named in it (GNA, in preparation) and, the path open, makes a *cold call*, the message to an agent with no shared logic variable (Section 3), which the inviter's social graph presents to its person as any request to connect, unless the inviter already knows the recipient, when the call opens a new channel between them and asks nothing; the link admits one first contact, and the befriending is the two persons' decision. Where no path opens within a minute of the redemption, the recipient's Grassroots App tells its person that the link could not connect them, and that they may try again on another network. The link, its registration and its redemption are the contract GRV of the rendezvous server, a ward of the person who runs it; and a person who chooses to be found lists themselves, by an email address it verified, with a directory, a second ward under the contract GDS, whose page gives whoever looks them up a link of theirs (GNA, in preparation). The Grassroots App carries the persons' acts of both: Invite

by link, List and Unlist. A person may also declare a web name that resolves to their public key, as grassroots personal websites provide (in preparation); anyone who has the name has the key.

Introductions via a mutual friend. A mutual friend Carol of Alice and Bob introduces them, GKK:3 [20, 51]: each agent presents the introduction to its person, on mutual consent the friend channel is established as in Section 4.1, Carol taking no further part, and if either declines no friendship forms. The mutual friend vouches for the identities of both, having a friend channel to each.

Whom the Grassroots App knows. The people Alice's Grassroots App knows are the agents it holds a channel with, and it comes to know one only by Alice's consent. A GLP program may send a variable in a message, so an end of a variable may reach Alice's Grassroots App from an agent it does not know, whatever program sent it; the global name of a shared variable carries the key of the agent at its other end [54, 59]. The runtime holds such an end, nothing sent on it and nothing accepted from it, and reports it [59]; the Grassroots App asks Alice whether to know that agent, and on her yes adds it to the agents it knows, attests with it (Section 8.1) and releases the end by `authorise_link/2`, and on her no drops the end. Acquaintance by GKK:2 and introduction by GKK:3 are the cases in which that consent is a clause's: Alice's yes to the act is her yes to knowing the agent. Two participants of a program converse once they know each other; the Grassroots App opens their conversation on the program's first message between them, as at an invitation (Section 5).

Reconnection. A friend channel outlives the path beneath it: a message to a friend with no path waits until there is one. It does not outlive a crash: the agent that restarts begins a new epoch, and the channel is widowed at both ends. The friendship stays: each agent, finding the channel widowed, drops it, and the one with the smaller key opens a new channel to the friend, as to any person it knows, with no volition elicited. A friendship made since the agent's last save is lost with its state; recovering it is the secure social graph's. When two agents become friends, each signs with `sign/3` the pair `attest(PkA, PkB)`, `PkA` the lexicographically smaller key, and gives the signature to the other. When an agent's address changes, its social graph distributes the new address to its friends; for a friend it cannot reach it presents the friend's signature to a rendezvous agent, the friend does likewise, the rendezvous agent checks that each request is signed by the key it names and that the two signatures are the two friends' over their pair, and relays to each the address of the other (GNA, in preparation); each punches, and the friend channel resumes on the path, with no volition elicited. The rendezvous agent checks keys, so an update of the Grassroots App leaves the signatures friends hold good. The runtime, not the caller, puts the source identity of the program whose execution signs into the signed term (Section 8), so a grassroots program cannot mint a social-graph attestation or an invitation link.

4.3 Identity and State Recovery

A participant may suffer a *major fault*: the loss of their private key, or the loss of their smartphone with the key retained. Grassroots platforms have no global resource to recover from, so recovery is peer-based, by friends serving as custodians: identity custodians, designated when the person chooses a key and carried on befriend, a supermajority of whom authorise a fresh key, which every friend of the old one renames to; and state custodians, every friend for the social graph, which checkpoint its friendships and restore them. The protocols, the fault model and the proofs are those of the secure social graph [15], and its GLP program, `secure_gsg`, extends `agent/4` with them. What the Grassroots App adds is the delegation: a grassroots program's participants are social-graph identities, so once the graph has replaced a lost key the program's own recovery proceeds against the renamed agent; a program supplies its own state custodians, chosen among the person's friends who run it, and its own read-back, as the currencies of Section 9 do. State recovery is privacy-preserving: a custodian's smartphone stores a friend's state without its person having access to it (A3 of Section 8). The Grassroots App runs the social graph without recovery first, and takes it up once the social graph and the store run (Section 9).

5 Dissemination of Programs

The contract. What the Grassroots App does with programs is the grassroots contract GPD, which extends GFP, the contract of items forwarded with provenance, whoever passes an item on adding their signature to it [20], and through it the social graph's, GSG (Section 4), and from which the Grassroots App's own program is compiled; its clauses are the acts of this section, and what the Grassroots App does by hand around the compiled contracts is the operating system, which Appendix A specifies. The text in the box is the repository's copy, `programs/social/graph/GPD.txt`.

GPD: Grassroots Programs Dissemination

GPD extends GFP [20].
A *program* is an item: compiled code, signed by its compiler, that gives the grassroots contract it is compiled from, its clauses and its speech-act schemas; its *developer* is its author. Alice, Bob and Carol are persons.

1. ◆ *Program Publication.* Carol may publish a program by signing it, attesting that its schemas say what its clauses say, that it is the result of compiling the schemas, and that it is expected to behave correctly according to the contract.

2. ■ *Developer Trust.* Alice may trust Carol, whom she knows, as a developer, and may withdraw her trust.
3. ■ *Program Offering.* Carol may offer a program she holds to her friend Alice by forwarding it to her.
4. ■ *Program Installation.* Alice may install a program she published, as well as a program offered to her whose developer she trusts.
5. ■ *Program Invitation.* Alice may invite her friend Bob to join her in a program she installed, with Bob's consent, Bob trusting its developer; Bob then has the program installed, with Alice's signature added, and each is a participant of the program with the other.
6. ■ *Leaving a Program.* Alice may leave a program she installed; it is then no longer installed, and her participation in it with every participant ends.

Interface pragmas

Programs: the programs Alice installed.
Developers: the developers Alice trusts.
Offers: the items offered to Alice.
Program Publication: `Carol [Publish] <program>`
Developer Trust: `Alice [Trust] [Distrust]`
Program Offering: `Carol [Offer]`
Program Installation: `Alice [Install own] [Install]`
Program Invitation: `Alice [Invite]`

```
   Bob: Alice invites you to join her in a
program.
```

Leaving a Program: `Alice [Leave]`

Implementation pragmas

Program Installation: before installing a program, Alice's app checks that its developer is one she trusts, that its schemas pass the checker, and that its compiler's certificate matches its code; the code is type-checked when activated.

A grassroots program is compiled from its contract. A developer, a person with a key, writes a grassroots contract: its clauses, their speech-act schemas and its pragmas [20]. The checker certifies that the schemas are syntactically grassroots, and the compiler of the cascade prints from the schemas and the pragmas the platform the contract specifies, with its screens, which compiles to GLP [28]; so the code of a grassroots program is the contract's own, and every word the program shows is in its contract. A *program* is the compiled code, signed by its compiler, which gives the contract it is compiled from: the compiled program carries its *source identity*, the hash of its source, its *compiled identity*, the hash of its compiled code, and the compiler's *certificate*, the compiler's key and its signature over the two identities [59, 60], and answers `contract(Text, Schemas)` with the contract it is compiled from, its clauses as text for a person to read and its speech-act schemas with the pragmas as the term the checker and the compiler read, and `version(Current, Back)` with its version and the earliest version of itself it is compatible with. The developer publishes the program by signing it, attesting that the schemas say what the clauses say, that the code is the schemas' compilation, and that it is expected to behave correctly according to the contract (GPD:1); a program so published is an item, and whoever passes it on adds their signature, so a program shows who published it and who passed it on [20]. A Grassroots App receiving a program makes the checks GPD's implementation pragmas state: its developer, the first signer, is one its person trusts (GPD:4), the schemas pass the checker, the compiler's certificate matches the code, and the code type-checks when activated (Section 3). The attestation's first part anyone judges by reading the clauses against the schemas, and its second anyone checks by compiling the schemas and comparing the compiled identities; a store's owner who lists a program signs it having done both (GAS:2).

Trusted developers. A person trusts the developers they choose, as they trust compilers (GPD:2). The Grassroots App keeps the keys of the developers its person trusts; a program is installed only from a developer its person trusts (GPD:4). A person comes to know a developer by their key, as from a store's page (Section 7), or by a friend's introduction (GKK:3), which a friend's invitation to that developer's program (GPD:5) is preceded by where the person does not know the developer; the Grassroots App then asks its person whether to trust them. A person withdraws their trust the same way; a program of theirs already installed stays until the person leaves it (GPD:6).

A grassroots program instance. An instance of a grassroots program is a set of people running the program on their own smartphone and connected via it. An instance begins with one person, who installs the certified program and activates it; others join it on invitation. A person joins an instance on the invitation of a friend who is a participant, and a program that provides for it may let two participants who are not friends converse (Section 4). Two instances coalesce when a member of one invites a friend who is a member of the other to join, and the friend accepts.

Publication by a developer. A developer publishes a program by signing it (GPD:1) and offers it to their friends (GPD:3), who pass it on to theirs, each adding their signature, so a program carries the signatures of its developer and of everyone who passed it on; the person installs it (GPD:4), and an instance begins when they activate it. A store is a developer whose agent runs on a server and offers the programs it holds (Section 7).

Distribution with an invitation. A grassroots program is distributed from friend to friend (GPD:5). Alice invites her

friend Bob from within the program: an act of its contract that names Bob sends him a message, and the program's first message to a friend it has no conversation with is its request to invite him, which it hands to the Grassroots App over its social-graph channel, naming Bob, and the message waits until the conversation opens. Alice's Grassroots App then offers Bob, over their friend channel, two things and nothing else, with her signature: the program's source identity, and one end of the *root channel*, the GLP channel over which their two executions of the program will converse, whose messages are of a type of the program's own, opaque to the Grassroots App [60]. The invitation is the same whether Bob has the program or not. Bob's Grassroots App looks the source identity up in its registry; where he lacks the program it asks Alice's Grassroots App for it, which ships it with her signature added (GPD:5), and checks it as above; only then does it ask Bob, who reads the contract and consents or declines, a program he declines being dropped. Where Bob declines, lacks the program and is not Alice's friend, or is not known to Alice's Grassroots App, the social graph tells the program so, which drops what it held for Bob; a later message to Bob asks again. A program thus arrives with three certifications: (1) by its developer, who attests that it is the compilation of the contract it gives; (2) by its compiler, whose certificate binds the compiled code to its source [20]; and (3) by Alice who invited Bob, whose invitation is signed over the source identity and Bob's key. To join, Bob signs a one-sided *undertaking* to keep the program's contract towards all its participants and to run its certified code [20], which his Grassroots App presents to each participant Bob first comes into contact with through the program, beginning with Alice.

Activation and joining. The Grassroots App keeps a *registry* of the programs installed on the phone, each with its developer, the GLP channels granted at installation, and whether it is running. The Grassroots App activates a program by posting its entry goal to the certified program, which is activated only if it type-checks (Section 3) [54, 60]. The Grassroots App holds no declaration of a program's types. It declares one *entry template*, a parameterised procedure whose arguments are the person's key and the two channels the Grassroots App grants, and whose two type parameters are the program's own types: what it draws for its person on the user-interface channel, and the messages of its root channels; every grassroots program declares its entry as the template's instance at its own two types; and the Grassroots App's procedures that handle a root channel or a user-interface channel pass it on and never inspect what it carries, so they are type-checked once for every program [60]. The Grassroots App activates a program with a budget it grants and a control stream it writes; both are the program's own clauses, the compiler having partially evaluated into its code the meta-interpreter that specifies them (Section 8.3). Each Grassroots App then hands its execution of the program its end of the root channel, Alice's the end it kept and Bob's the end the invitation carried; the *conversation* between the two executions consists of the root channel and the variables occurring in terms communicated on it. Before any traffic flows on it the two Grassroots Apps run the handshake of Section 8.1. An invitation is looked up in the registry: a program not running is activated and granted its two channels, and where it runs already the new conversation's root channel goes to that execution, so an invitation between members of two instances coalesces them.

The friend channel is the medium of distribution: the offer of a program, the request for a program the invitee lacks and the shipped certified program, the root channel's end and the two identities of the handshake all travel over it, as messages of the social graph program, and the social graph signs the invitation; a program asks for an invitation over its social-graph channel . The social graph keeps, for each program, the set of participants the person converses with in it.

6 The Programs

Every other grassroots platform reaches a person by dissemination (Section 5), as a program compiled from its contract: social networking, GSN, and social networking with provenance and with AI agents, GSNP and GAI [51, 56]; child-safe social networking, CSSN [55]; coins and bonds, GC and GB [49, 57]; and feeds and followers, barter and ride-hailing (in preparation). Each takes from the Grassroots App its two channels and nothing else, converses with the executions of itself at other participants over the conversations the social graph opens, and keeps its own state, with its own custodians where it has them; Section 9 says which run on the prototype.

7 The Grassroots App Store

A *store* is a ward of its owner, as a rendezvous server is (GNA, in preparation): an agent with a key of its own, run on a machine, holding a list of programs; what a person does with a store is the contract GAS, which composes GPD (Section 5) with GSV, the contract of services, which instantiates the contract of guardians and wards [20] (GNA, in preparation), restricts the composition, and extends it; the text in the box is the repository's copy, `programs/social/graph/GAS.txt`. The *Grassroots App Store*, at https://peoplesweb.org, is the store of the GLP repository, in its directory `servers/peoplesweb`; the site, the server and the files that deploy them are in the repository, and anyone can set up a store (GAS:1) and run one from them.

GAS: Grassroots App Store

GAS composes GPD with GSV (GNA, GLP Networking API and Services, in preparation), restricts the composition, and extends it.
A *store* is a service holding a list of programs. Alice, Bob and Carol are persons, Dave a ward.

1. ■ *Store.* Carol may make her service Dave a store.
2. ○ *Listing.* Carol may list in Dave a program developed by Bob, after adding her signature to its signatures, as long as she trusts Bob as a developer; and may delist it.
3. ■ *Store Trust.* Alice may trust Dave, Carol being known to her, and may withdraw her trust.
4. ■ *Installation from a Store.* Alice may install a program listed in a store she trusts.

Interface pragmas

Stores: the stores Alice trusts.
Listings: the programs listed in Carol's store.
Store: `Carol [Open store]`
`Dave: Carol asks you to be a store.`
Listing: `Carol [List] [Delist]`
`Dave: Carol asks you to list or delist a program.`
Store Trust: `Alice [Trust store] [Distrust store]`
Installation from a Store: `Alice [Install from store] <program>`

Implementation pragmas

Store: the store's page shows its key, its owner and its listings, each program with its contract, the checker's verdict and the artefacts of its cascade; Dave answers anyone who asks for a listed program.

Trust in a store is trust in its owner's choice of programs: the owner lists a program while they trust its developer, signing it (GAS:2), so the store's owner, and not each person, judges the developers; a person who trusts no store installs from the developers they trust themselves (GPD:4).

What the site shows. The site shows every grassroots contract the repository records: its clauses, the speech-act schemas, the checker's verdict, its pragmas, its developer, its paper, and its relation to the contracts it is written from, as GSG extends GKK and GSN amends GSG [20]; the contracts are drawn on a globe, GKK at the pole and a ring per depth of extension. For each contract it shows the artefacts the abstraction cascade produces from it, one run of the cascade recorded in the repository under `cascade/<NAME>/`: the clauses, the schemas with the verdict, the schemas in notation, the volition-guarded transactions, the volitional program the compiler prints from the schemas and the pragmas, its compilation to GLP, the log of a run with scripted persons, and the platform's interface [20, 28, 34]. And it runs each program live, with four persons, Alice, Bob, Carol and Dave, each an agent in an isolate of its own on the server, their messages carried between the isolates as the networking layer's simulation carries them; a visitor acts for each person in turn and sees what passed between the agents on the network. The live programs run on the server so that a visitor needs to install nothing; in use, each agent runs on its person's own smartphone.

The store's agent. The store's agent is compiled from GAS and runs on the server over the networking layer with no person and no screen, under the ward's key, which the site shows with its owner's. A person comes to know the store by that key (GKK:2), by a cold call over the Internet (Section 4.2), which the store answers for anyone; the store is nobody's friend, and its channel is unattested, as a rendezvous server's is (Section 8.1). A program may travel over an unattested channel, its integrity resting on its signatures and not on the channel. The Grassroots App Store's owner is the developer of the programs of the repository, publishing each (GPD:1) and listing it (GAS:2), so a person who trusts the store (GAS:3) installs any of them (GAS:4).

Installing from the store. A person who knows the store's owner trusts the store (GAS:3) and picks a listed program; their Grassroots App receives the program, checks it as Section 5 says, and installs it (GAS:4); the person activates it, beginning an instance of one, and invites friends (GPD:5), who receive the program from them and not from the store. Installing from a store and from a friend thus differ in one thing: whether the program comes with an invitation.

Publishing. A developer who is not a store publishes a program the same way, offering it to the people who know them (GPD:3), and they pass it on friend to friend (GPD:5); a store lists a developer's program when the owner holds it and trusts the developer (GAS:2), and a program installed from a store carries the owner's signature with the developer's. No store admits, reviews or withdraws a program on a person's smartphone: what a person installs is decided by the developers and the stores they trust and the friends who invite them, and what they run is decided by them.

8 Security

8.1 Attestation in Place of an Operator

A person's smartphone is controlled by that person, so no participant can establish by inspection that the smartphones of the others run the program. We assume three properties of the platform, realised in hardware by a secure element or a trusted execution environment [14]; they are assumed, not proved, and are the trust root of the design.

A1 (Attestation.) The operating system of each smartphone provides remote attestation of the application it runs: upon request, and bound to a given communication channel, a smartphone produces an attestation that it runs the

application published under a given developer key, by its name and version, which a peer can verify; if verification succeeds then the smartphone runs that application on that channel [1, 4, 14]. The binding is to the channel's key exchange, so an attestation for one channel cannot be replayed onto another. We trust the developer key to sign one build per name and version.

A2 (Cryptography.) The hash function is collision-resistant and the signature scheme is unforgeable, so distinct sources and distinct compiled codes have distinct hashes, and hash equality is identity. A channel established by key exchange between two key pairs is mutually authenticated and confidential.

A3 (Isolation.) The platform isolates the attested application's storage and keys from every other application and from the person, so that what the application persists is readable and writable only through the application itself [4].

The application an attestation names is the Grassroots App, which includes the virtual machine, the interface bridge, the networking layer, which holds the person's private key, and the social graph; of it we require three guarantees.

G1 (Execution.) The virtual machine executes compiled GLP correctly [59]; a trusted compiler compiles GLP source correctly, every compiled program it produces carries the source identity, the compiled identity and the certificate of Section 5, and it certifies no program that calls an OS-privileged predicate (Section 2); the machine activates no program whose certificate does not verify; and it holds the state of the programs it executes only within the Grassroots App, sealing what it persists.

G2 (Variables.) Every value delivered on a shared variable was assigned by the agent holding the writer, and no variable is assigned more than once, restarts included.

G3 (Conversations.) No traffic of a program is delivered on a conversation before that conversation's handshake has succeeded, and none between two participants before each has presented its signed undertaking to the other. An end of a variable that reaches an agent from an agent it does not know is *held*: nothing is sent on it and nothing is accepted from its holder, until the person consents to knowing that agent and the two have attested (Section 4.2). Upon restart every conversation is treated as crashed until a fresh attestation re-establishes it .

The runtime gives a program three predicates [54]: `sign(T, K, S)`, which assigns `S` the signed term of `T` under the key `K` and succeeds only where the runtime holds the private half of `K` for its own person, and puts into `S` the source identity of the program whose execution called it, which the caller neither supplies nor can choose; `signature(S, R)`, which assigns `R` the term `signed(K, H, T)` — the signer, the source identity of the program whose execution signed, and what was signed — where `S` is a signed term whose signature verifies, and the constant `unsigned` where it is not, so that a caller discriminates by matching and refuses a forgery instead of failing on it; and `decompose_module(P, K, Src, Cmp)`, which assigns the key of the person who compiled the program `P` and its two identities.

Fault and threat model. Smartphones may crash and the network may delay or lose messages; no synchrony is assumed. Persons may be arbitrarily malicious but act on the system only through their smartphones, and may restart a smartphone on an earlier version of its sealed state. The adversary may operate any number of persons and of machines not running the Grassroots App, and may attempt arbitrary protocol traffic, but cannot break A1, A2 or A3.

Mechanisms. Two agents that come to know each other, by GKK:2 in any of the three ways of Section 4.2, by GKK:3 or by the person's consent to a held end (Section 4.2), end with a mutually authenticated channel between them (A2); over it each requests, receives, and verifies the other's attestation that the peer's smartphone runs the Grassroots App, bound to the channel (A1). If either verification fails the channel is abandoned; otherwise it is *attested*, and the two know each other; befriending, GSG:1, is adopted over it. Every conversation of a program is created over an attested channel between two agents that know each other (Section 5). Every conversation opens with the *handshake*: before any traffic flows, the two Grassroots Apps exchange the program's version and the earliest version it is compatible with, `version(Current, Back)` (Section 5), and the root channel's type identity, the type identity of the entry template's opaque instance at the program's types (Section 3); they go on only if the two ranges intersect and the type identities are equal, and, where the two participants have not been in contact through the program before, exchange their undertakings. Upon disagreement, or upon any traffic on a conversation with no successful handshake, the Grassroots App aborts the conversation and delivers nothing: to the program it is silent, as a crashed peer is. Two invariants hold: every friend channel and every conversation of a program runs between two attested Grassroots Apps, and nothing flows on it before that holds; and the networking layer makes no social decision. The layer (GNA, in preparation) states two more, that an attestation is bound to the session it is presented on, and that the layer verifies attestations while the social graph decides what an unattested peer may be used for, a rendezvous server being unattested by design. The Grassroots App has a version, which its attestation carries, and an earliest version it is compatible with, a constant of its program; on the handshake of a friend channel each tells the other the earliest version it is compatible with and reads the other's version from its attestation, and the two go on only if the two ranges intersect. The rendezvous agent checks the keys that sign a request (Section 4.2).

Fix a program M and a set of agents P, each pair of communicating participants connected by an attested channel, each participant having joined M at a version compatible with every other's, and every Grassroots App granting M its two GLP channels under the grant discipline of Section 2. The execution of M at a participant p, together with what p's Grassroots App assigns on what it granted M, is *M at p*.

Two things then hold of the traffic carrying M's source identity. *Each participant runs M as written.* To have joined M is to hold a certified program with the source identity of a version of M, verified before installing against the certificate of a compiler its person trusts: by G1 the code is a correct compilation of that source and the machine executes it correctly, and by A2 hash equality is identity, so the source is that version of M. *The traffic of M passes between executions of M.* A conversation is created only over an attested channel, which by A1 and A2 only a smartphone running the Grassroots App holds; before any traffic flows, the two Grassroots Apps compare their versions of M and the root channel's type identity and exchange their undertakings, and where the versions are incompatible or the type identities differ deliver nothing (G3); each then hands its end of the root channel to its own execution of M and to nothing else; and thereafter a reduction (Section 3) of M holds only variables it held, variables it created, and variables arriving in a term assigned to a reader it holds, which by G2 was assigned by the holder of the paired writer — an execution of M, or a Grassroots App assigning under the grant discipline. A machine not running the Grassroots App holds no attested channel, and so no conversation of M.

The two together give a crash-failure model. A person may be arbitrarily malicious and acts only through a smartphone, which if attested runs M as written and if not holds no conversation of M, so what a participant observes is a behaviour of M under crash faults and network asynchrony; a restart on an earlier version of the sealed state is a crash and a recovery with loss of state, since G3 treats every conversation as crashed until a fresh attestation and G2 lets no variable be assigned twice. This is what the recovery of Section 4.3 presupposes [15]. And since the first contact between two participants carries their undertakings, every participant of an instance of two or more has signed the contract and another participant holds the signature. Thus, Proudhon's condition (5), freely accepted and signed by all the participants, is enforced by the code [20].

8.2 Separation of Programs

All goals of one agent run in one heap, so two programs on one smartphone are not separated by an address space; they are separated by reachability: every variable has one writer occurrence and one reader occurrence [54], a program obtains a variable only by being given one, and the Grassroots App gives a program its two GLP channels and no more.

The *holdings* of a goal are the variables occurring in it, and the execution of a program M at an agent is the goal with which the Grassroots App activated M there together with the goals descended from it by reduction. A reduction adds to those holdings only variables it creates and variables occurring in a term assigned to a reader they hold: matching binds the head's variables to subterms of the goal's arguments, the remaining variables of the body are fresh, and no guard, system predicate or body kernel of GLP returns a variable its goal does not hold [54]; a certified program calls no OS-privileged predicate (Section 2).

What a program holds therefore never grows beyond what it was given and what it makes. At activation it holds one end of each of the two granted GLP channels, whose other ends its Grassroots App holds; on the social-graph channel it receives root-channel ends whose other ends are held by executions of the same program at other participants; and on the two the Grassroots App assigns, under its grant discipline, only ground terms, fresh variables whose other ends it holds, and those root-channel ends. Two distinct programs thus hold no variable in common, on one smartphone or across smartphones; a program communicates with the Grassroots App only on the two channels it was granted; and an execution of M at another agent sends only messages of M.

8.3 Resource Bounds, Control and Failsafe Execution by a Meta-Interpreter

Separation bounds whom a program reaches; it does not bound how much of the smartphone a program uses, it gives the person no way to stop a program that misbehaves or does not terminate, and it does not keep the failure of a goal of a program from reaching the Grassroots App. The Grassroots App provides both as the Logix system provided the functions of an operating system for Flat Concurrent Prolog [24, 61]: Logix ran a program in one of four modes, trust, failsafe, control and interpret, each but trust specified by a meta-interpreter [46] and implemented by a transformation of the program realising the partial evaluation of that meta-interpreter. Here each function the Grassroots App performs on a running program is specified as a meta-interpreter of the program, which the compiler partially evaluates with respect to the program, so that nothing is interpreted at run time. The Grassroots App has three such functions, resource bounds, control and failsafe execution.

The budget counts reductions of the program (Section 3). The meta-interpreter `execute(Goal, Budget, Requests, Control)` reduces `Goal` with at most `Budget` reductions; when the budget is spent it requests more on `Requests`, a stream the Grassroots App reads, and waits; and it reads `Control`, a stream the Grassroots App writes, and ends every goal of the program once `abort` is written on it, a goal waiting for its data included. A conjunction splits its budget between its conjuncts, merges their request streams, and distributes

the control stream to both (`merge/3` and `distribute/3` are in Appendix C):

```
execute(true, _, [], _).
execute((A, B), N, Rs?, Cs) :-
    N1 := N? // 2, N2 := N? - N1?,
    distribute(Cs?, Cs1, Cs2),
    execute(A?, N1?, Rs1, Cs1?),
    execute(B?, N2?, Rs2, Cs2?),
    merge(Rs1?, Rs2?, Rs).
execute(G, N, Rs?, Cs) :-
    otherwise, unknown(Cs?), N? > 0 |
    N1 := N? - 1, reduce(G?, G1, B, Ok),
    resume(Ok?, G1?, B?, N1?, Rs, Cs?).
execute(G, N, [more(N1)|Rs?], Cs) :-
    otherwise, unknown(Cs?), N? =?= 0 |
    execute(G?, N1?, Rs, Cs?).
execute(_, _, [], [abort|_]).

resume(reduced, _, B, N, Rs?, Cs) :-
    execute(B?, N?, Rs, Cs?).
resume(failed, G, _, _, [failed(G?)], _).
```

`reduce(G, G1, B, Ok)` assigns `Ok` `reduced` and `B` the user goals of a clause of the program whose head matches `G` and whose guard holds, its system goals executed within it, and assigns `Ok` `failed` where the program has no such clause [46]; where it fails it hands the goal back as `G1`, so that `resume/6` can report the failed goal without the reducing clause reading `G` twice, which the single-reader restriction forbids [54]. A goal of a program therefore does not fail: the Grassroots App reads `failed(G)` on the request stream, that goal is abandoned, and the rest of the program proceeds. `unknown(Cs)` holds while nothing has been written on `Cs` [54]. A goal whose budget is spent sends `more(N1)` with `N1` unbound and proceeds with whatever budget the Grassroots App assigns; the Grassroots App assigns the budgets of a program at the rate it sets for it, and a program whose request it does not answer stays suspended. A program *terminates cleanly* when every goal of it has terminated: its request stream closes with nothing written on its control stream. The person stops a program through the Grassroots App user interface: the Grassroots App writes `abort` on the control stream, every goal of the program ends and its request stream closes, and the Grassroots App closes the program's conversations, whose peers see a crash (Section 8.1).

Partial evaluation. The compiler will remove the interpretation, by partially evaluating `execute/4` with respect to the program [26, 46]; the code it certifies is then the residual program. This is ongoing work and no open problem, Logix having compiled its four modes exactly so [61]; the residual programs below and in Section 9 were derived by hand. In the residual program the program's own clauses carry the budget and the control stream and test them. Since the control test precedes a clause's own guard, a goal waiting for data is ended by an abort as a running one is; the meta-interpreter as listed cannot end it, since a GLP guard admits only guard predicates — the built-in guards and the unit clauses the compiler unfolds into term matching — and `reduce/4`, having a body, is therefore a body goal, committed to before the goal's data arrive, so the listing states the semantics and the residual program realises it. What remains at run time is, per reduction, a test that the control stream is unwritten, a comparison and a subtraction. The distributes are there because a reader occurs once in a GLP clause [54]; a typed relaxation admitting several readers of a ground stream within one agent would remove them, and is future work.

Partially evaluating `execute/4` with respect to a program yields, for each of its predicates, a predicate with three further arguments. For a stream counter

```
count([_|Xs], N, [N1?|Out?]) :-
    N1 := N? + 1, count(Xs?, N1?, Out).
count([], _, []).
```

the residual program is

```
count([_|Xs], N, [N1?|Out?], B, Rs?, Cs) :-
    unknown(Cs?), B? > 0 |
    B1 := B? - 1, N1 := N? + 1,
    count(Xs?, N1?, Out, B1?, Rs, Cs?).
count([], _, [], B, [], _) :- B? > 0 | true.
count(Xs, N, Out?, B, [more(B1)|Rs?], Cs) :-
    unknown(Cs?), B? =?= 0 |
    count(Xs?, N?, Out, B1?, Rs, Cs?).
count(_, _, [], _, [], [abort|_]).
```

The first clause charges one reduction, the unification being in the head, and passes the remaining budget to the recursive call; a body with one user goal needs no merge and no distribute. Each predicate gains, beside these, a failure clause taken when no clause of the program matches, which reports the goal on the request stream and closes it. The third clause is the request clause and the fourth the abort clause, which applies whether the goal was running or waiting for its stream and closes the goal's output. What remains of the meta-interpreter at run time is the control test, the comparison and the subtraction in each guard.

A program is *budgeted* if the Grassroots App activated it as `execute(Goal, n, Rs, Cs)` for a budget n and assigns every `more(N)` on `Rs`; the k-th such assignment is its k-th *grant*, of budget n_k. Take the sum, over the `execute/4` goals of its execution whose budget argument is bound, of that argument. Activation sets it to n and the k-th grant adds n_k; splitting a budget between two conjuncts leaves it unchanged, as do the halt, abort and request clauses; and the reducing clause, which fires only where the budget is positive, performs one reduction of the program and takes one from it. So it grows only by a grant and never falls below zero, and before its k-th grant a budgeted program has performed at most $n + n_1 + \cdots + n_{k-1}$ reductions — of the compiled program as of the source, since by G1 the one is the correct compilation of the partial evaluation of `execute/4` with respect to the other. A reduction allocates at most the cells

of one clause body, creates at most the goals of one clause body, and assigns at most the variables of one clause body, so the heap a program allocates, the goals it creates and the messages it sends are each bounded by a constant of the program times its reductions; the budgets the Grassroots App grants bound all four. A program cannot exhaust the smartphone's resources at the expense of the Grassroots App or other programs, the person can end one that misbehaves, and no failure within one reaches the Grassroots App. The prototype does not yet activate its programs under the meta-interpreter (Section 9).

8.4 The Three Certifications

The clauses of a grassroots contract are of three legal types [20]: a clause is *enforced* if no act of the contract can breach it; *attested* if a party may breach it and the contract records the breach in another party's hands; and *undertaken* if the contract records who undertook it and its performance is outside the contract.

A grassroots program arrives with three certifications (Section 5). The developer attests that the program is the compilation of the contract it gives; the receiving Grassroots App runs the checker on that contract, so a program it installs is grassroots by construction [20]. The compiler being trusted, its certificate says further that the code is a correct compilation of a GLP program calling no OS-privileged predicate (Section 2); the programs of Section 9 call none. Once the compiler partially evaluates the meta-interpreter into a program, the certificate will say that too. What Sections 8.2 and 8.3 establish holds of such code: a program cannot harm the Grassroots App or other programs, whatever its developer intended, and cannot breach an enforced clause of its contract. The inviting friend runs the program on an attested smartphone and chose to invite; the invitation provides some comfort and no guarantee. In a party's undertaking the clause to run the certified code is undertaken [20]: its performance is outside the contract, and no run shows whether a party keeps it. Here attestation enforces it: the handshake proves that both parties run the certified code, and a party that does not run it has no conversation. The Grassroots App accepts a compiler's certificate when its person trusts the compiler's key, and a program when its person trusts its developer's key (Section 5); the compiler bundled with the Grassroots App is trusted by installing it, and the developers a person trusts are their own choice, the store's key among them when they add it.

9 Programs on the Prototype

The Grassroots App prototype runs on a physical smartphone and on an iPhone simulator. The programs compiled from the contracts GKK, GSG, GSN and GC run on it on the simulator, one instance of the Grassroots App running the program of every agent, the agents communicating over

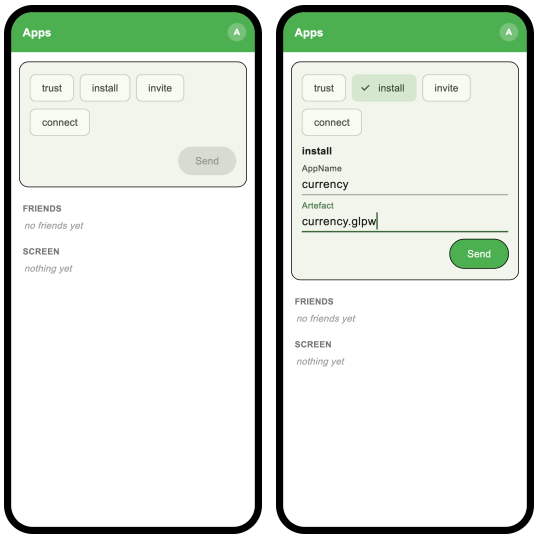
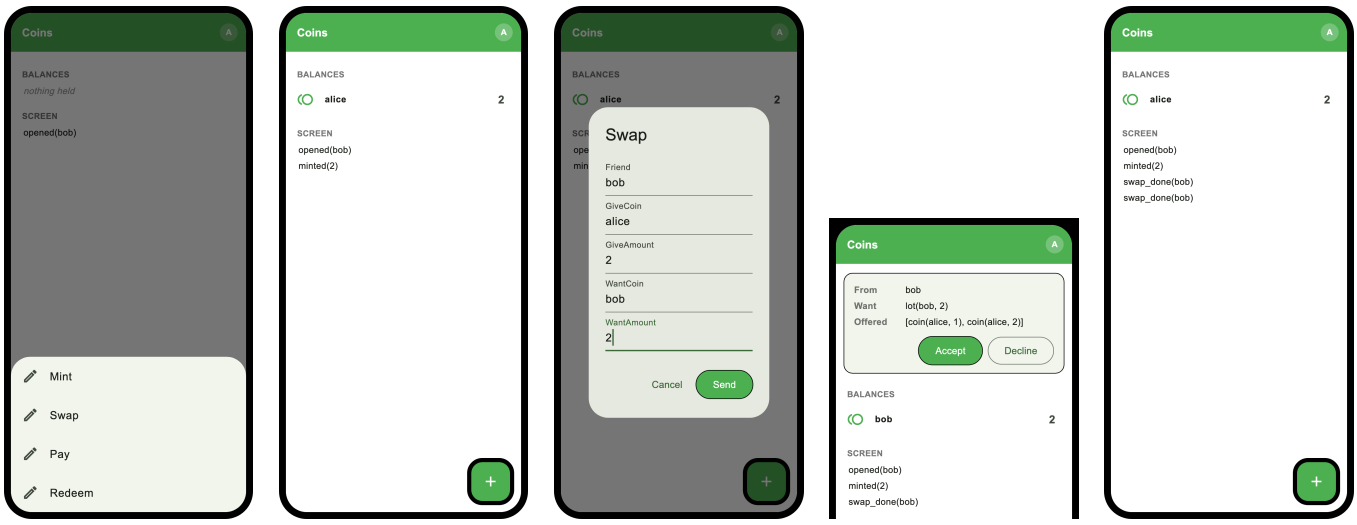


Figure 2. The Grassroots App's own person interface: the four acts a person wills of their Grassroots App — trust a compiler, install a program, invite a friend, connect — as a menu, and the Install form, with `currency.glpw` and the name the program takes. Every display element is the default widget of the type of a question the Grassroots App's program asks its person [28].

channels within it [20, 28]; two platforms in an earlier, handwritten form ran on a physical smartphone, each given below by what it takes from the Grassroots App and what it supplies.

Social networking, including child-safe. Grassroots social networking [51] takes GLP channels to the executions of itself on friends' smartphones and the user interface, and supplies personal feeds, with authors and followers who create and store them, and a dissemination protocol in which communication occurs only along the edges of the social graph. Child-safe social networking [55] adds a relation the social graph does not have, between a parent and their child, in which the parent's volition governs whom the child befriends within the program; the Grassroots App knows nothing of it, it being clauses and guards in the program's own program.

Grassroots currencies. Grassroots currencies [33, 49, 57] take GLP channels to the executions of themselves on friends' smartphones, over which coins and bonds pass between friends, and the social graph's recovery of identity. Each person is the sovereign of their own currency, and every transaction in their coins requires their participation, so the sovereign's log is a single-writer total order. The program supplies that log and the state custodians that restore it: a designated subset of the sovereign's friends participate in every transaction, finality is coupled to a supermajority of them, and a recovering sovereign reads the log back from them, so it comes back exactly [15].



(a) The four acts (b) After a mint (c) The Swap form (d) Bob's offer (e) After Accept

Figure 3. The currency program, on Alice's smartphone, in one conversation with Bob. (a) The panel's "+": Mint, Swap, Pay and Redeem, the program's four request clauses, each perpetually pending. (b) After minting two coins: `BALANCES alice 2`, and the screen carrying `opened(bob)` and `minted(2)`. (c) The Swap form, its five fields the five writers of its clause's question, in the question's order. (d) After the swap, a further offer from Bob awaiting Alice's volition. (e) After accepting it.

Implementation status. Implemented and run: the secure social graph with befriending, unfriending, checkpoints and restoration, and the replacement cascade; the Grassroots App on two smartphones, each with its own key pair, which befriend, read a certified program's artefact and verify its two certifications before installing it, invite, exchange undertakings signed under their two keys, and open the root channel by `run/3` at each end with the two grants; the compiler's certificate and refusal; the meta-interpreter with budgets and control, run interpreted, its failsafe reduction specified and not run, its partial evaluation into the program being ongoing work, the residual programs measured having been derived by hand. Both platforms above run as certified programs of the Grassroots App, in one heap and on two smartphones: each is installed after its certificate and its creator's certification verify, and activated by `run/3` with its two grants at the handshake; a swap of the currency's own contract settles between two participants over the conversation the social graph delivered, and under the child-safe platform two participants become friends and a message passes. Figure 2 shows the Grassroots App's own interface, Figure 3 the currency program in use, and Appendix D further screens of the Grassroots App. Each platform kept its own contract and its own program; what changed in each is that it takes its conversations from the social graph in place of a network entry of its own. The warm call runs end to end on three smartphones: the held end is reported to the program, the introduction runs, the two participants befriend, each end of the held link is released by its own person's volition, and the value reaches the execution the link is anchored at. Both platforms also run together under one Grassroots App, in one heap and on two smartphones: each person installs both, each program has its own user-interface grant and its own conversations, and in a single run the swap of the currency's contract settles and the child-safe platform's message arrives, the two protocols proceeding over the one friendship. The rendezvous server runs on the store's host, https://peoplesweb.org, since 2026-11-07, over the networking layer, with the social graph's source identity. Not implemented: the shipping of a program to an invitee who lacks it; the compilation of a shipped program by the receiving Grassroots App, which installs the compiled code it receives; the developer's warranty in the form of Section 5, the prototype's checking a creator's certificate carrying the verdict `grassroots` where the checker prints `syntactically_grassroots` and its variants; the developers a person trusts; the store's agent and installation from a store; the invitation link and the reconnection through a rendezvous server; and the activation of a program under the meta-interpreter — `run/3` posts its goal with no budget and no control stream; the meta-interpreter, its residual program and a granting Grassroots App run on their own in `programs/social/graph/budget`.

Measurements. The runtime was measured on a laptop rather than on the phone — an Apple M3 under macOS 15.6, Dart 3.11.0 — ten runs each, every run a fresh process, with the load time subtracted per process. A goal record costs about $23 + 7n$ words for a goal of arity n, some 50 words for the social graph's `agent/4`, and a variable pair about 17 words while unbound; the BEAM's newly spawned process is 327 words (Table 2). The handshake, from the person's invitation

to the root channel's end reaching the invitee's program, with the two social-graph agents in one heap, takes 13.3 ms (standard deviation 1.6). The stream counter of Section 8.3 over 50,000 elements runs unbudgeted in 0.95 s (0.02), at most about 2.6×10^5 goal reductions per second; as the residual program under one grant covering the run it takes 1.30 s (0.02), so the budget test and decrement cost 36 per cent; under grants of 1,000 reductions, 50 requests, it takes 1.31 s (0.02), and under grants of 10, 5,000 requests, 1.40 s (one run), so a request round trip costs about 0.02 ms. Interpreting the counter through `execute/4` as Section 8.3 lists it takes 2.19 s (0.04), 131 per cent over the unbudgeted run, the time about doubling with the input: one run at each of 500, 1,000, 2,000, 4,000 and 8,000 elements takes 0.018, 0.035, 0.066, 0.146 and 0.319 s. The interpreter the prototype runs, which omits the failsafe reduction, takes 1.95 s (0.04), 105 per cent over. Partial evaluation thus takes the cost of the budget from 131 per cent to 36. The heap a program holds at rest on the phone is not yet measured.

Limitations. Grassroots personal websites (in preparation) do not yet run as a program, and hosting a community that decides by consensus is future work. The separation of programs rests on the correctness of the trusted compiler and of the virtual machine, not on hardware isolation: the programs of one person share one heap (Section 8.2). The attestation of A1 is provided by the smartphone's operating-system vendor, a third party the design otherwise avoids (Section 8.1).

10 Related Work

Super-apps and app stores. A super-app's operator admits, distributes and withdraws mini-apps, grants and revokes their capabilities, and holds the accounts through which people are identified [2, 3, 69]; the app store of a smartphone's operating system admits an app after review, signs it, and can withdraw it from every phone; here there is no operator, a store is a developer among others, and what a person installs is decided by the developers they trust and the friends who invite them (Sections 5 and 7). Appendix B compares the two over the architecture, the thirteen security mechanisms and the ten threats catalogued for super-apps [69–71].

The BEAM. Erlang and GLP descend from the same work. Erlang began as an experiment in adding concurrency to Prolog, built on a Prolog meta-interpreter, and the concurrent logic programming languages — Parlog, KL/1 and Strand — were read afterwards, for inspiration; Armstrong records that this turned out to be a mistake, the concurrency of those languages being implicit and fine-grained where Erlang's is explicit and coarse-grained, and reports an Erlang-to-Strand compiler abandoned because Strand created far too many processes [5]. Erlang gave up the shared logic variable and adopted message passing by copying, for the sake of error handling [5]. GLP is the branch that kept the shared logic variable, under the single-occurrence and single-reader single-writer restrictions that make it assignable once and sendable across a network [54, 59], and the family it belongs to is the one Erlang set aside [13, 38, 48, 65]. An Erlang cluster is not a grassroots platform: a node carries a host name, is found through a port mapper daemon running on that host, and joins a cluster whose members share one cookie and connect to one another by default (Table 2). Each of those is a resource outside the network that the running system requires, which is what bars a platform from having two instances that operate independently [34, 53]. The distribution is built for one administrative domain and says so: the documentation's own warning is about an attacker who reaches the whole cluster through one node [17].

Isolation in one address space. Running mutually suspicious programs in one address space and isolating them by the language is the design of the software-isolated processes of Singularity [25], whose channels pass a message by transferring its ownership rather than by copying it [18], of the type-safe kernel extensions of SPIN [8], and of KaffeOS, which accounts resources to each Java process of one runtime [7]; in the object-capability model a program's authority is what it can reach [39], and a WebAssembly module reaches only its own linear memory and the imports it was given [22]. Here the isolation is a theorem over the single-occurrence variables of the language (Section 8.2), and the resource bound is a meta-interpreter the compiler partially evaluates into the mini-app, where a blockchain machine charges gas per instruction [68].

Decentralised and federated social networking. ActivityPub and Mastodon [41, 44, 45], Solid [35], Decentralized Identifiers [66], Matrix [37] and Bluesky's AT Protocol [31] require servers with domain names, so an operator remains essential to their users; in the taxonomy of [53] they are federated, where Scuttlebutt [64] and the Grassroots App presented here are grassroots, and none hosts another platform.

Key recovery. Scuttlebutt [64] and Nostr [40] have no recovery from a lost key; threshold schemes [47] reconstruct the secret from shares, and social-recovery wallets [10] replace the key on a blockchain, a global resource. Here custodians authorise a fresh key and friends carry the rename [15].

Attested execution. Remote attestation [1, 4, 14] has been used to reduce a protocol's fault model from Byzantine to crash [12, 32]; G2 and G3 answer sealed-state rollback [29, 36, 62] and the forking of an attested application [9].

References

[1] Abdulla Aldoseri, Tom Chothia, José Moreira, and David Oswald. 2023. Symbolic Modelling of Remote Attestation Protocols for Device and App Integrity. In *ASIACCS '23: Proceedings of the 2023 Asia Conference on Computer and Communications Security.*

[2] Alibaba Cloud. 2026. Build Cross-Platform Apps with SuperApp Open Platform. https://www.alibabacloud.com/help/en/superapp/superapp-

bap-public-intl/product-overview/introduction-to-superapp. Super-App Business Application Platform documentation; accessed 30 July 2026.

[3] Alipay. 2026. Alipay International Mini Program Development Platform: Documentation. https://miniprogram.alipay.com/docs/. Accessed 30 July 2026.

[4] Apple Developer Documentation. 2024. Implementing App Attest. https://developer.apple.com/documentation/devicecheck/implementing_app_attest. https://developer.apple.com/documentation/devicecheck/implementing_app_attest.

[5] Joe Armstrong. 2007. A history of Erlang. In *Proceedings of the Third ACM SIGPLAN History of Programming Languages Conference (HOPL-III), San Diego, California, USA*, Barbara G. Ryder and Brent Hailpern (Eds.). ACM, 1–26. https://doi.org/10.1145/1238844.1238850

[6] Joe Armstrong. 2010. Erlang. *Commun. ACM* 53, 9 (2010), 68–75. https://doi.org/10.1145/1810891.1810910

[7] Godmar Back and Wilson C. Hsieh. 2005. The KaffeOS Java Runtime System. *ACM Transactions on Programming Languages and Systems* 27, 4 (2005), 583–630. https://doi.org/10.1145/1075382.1075383

[8] Brian N. Bershad, Stefan Savage, Przemyslaw Pardyak, Emin Gun Sirer, Marc E. Fiuczynski, David Becker, Craig Chambers, and Susan Eggers. 1995. Extensibility, Safety and Performance in the SPIN Operating System. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 267–284. https://doi.org/10.1145/224056.224077

[9] Samira Briongos, Ghassan Karame, Claudio Soriente, and Annika Wilde. 2023. No Forking Way: Detecting Cloning Attacks on Intel SGX Applications. In *39th Annual Computer Security Applications Conference (ACSAC)*. https://www.acsac.org/2023/program/final/s207.html

[10] Vitalik Buterin. 2021. Why we need wide adoption of social recovery wallets. https://vitalik.ca/general/2021/01/11/recovery.html.

[11] Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. 2020. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, Vol. 2781. CEUR-WS, 51–60. https://ceur-ws.org/Vol-2781/paper5.pdf

[12] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. 2007. Attested Append-Only Memory: Making Adversaries Stick to their Word. *ACM SIGOPS Operating Systems Review* 41, 6 (2007), 189–204.

[13] Keith Clark and Steve Gregory. 1986. PARLOG: parallel programming in logic. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 8, 1 (1986), 1–49. https://doi.org/10.1145/5001.5390

[14] George Coker, Joshua Guttman, Peter Loscocco, Amy Herzog, Jonathan Millen, Brian O'Hanlon, John Ramsdell, Ariel Segall, Justin Sheehy, and Brian Sniffen. 2011. Principles of Remote Attestation. *International Journal of Information Security* 10, 2 (2011), 63–81. https://doi.org/10.1007/s10207-011-0124-7

[15] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. Securing People and their Machines Against Major Faults (Full Version). *Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear* (2026). https://arxiv.org/abs/2607.02304 arXiv:2607.02304.

[16] Ericsson AB. 2026. Erlang/OTP Efficiency Guide, Version 29.0.6. https://www.erlang.org/doc/system/eff_guide_processes.html. Chapters "Processes" and "Memory Usage"; accessed 4 September 2026.

[17] Ericsson AB. 2026. Erlang/OTP System Documentation, Version 29.0.6. https://www.erlang.org/doc/system/distributed.html. Chapters "Distributed Erlang", "Processes" and "Compilation and Code Loading"; accessed 4 September 2026.

[18] Manuel Fähndrich, Mark Aiken, Chris Hawblitzel, Orion Hodson, Galen Hunt, James R. Larus, and Steven Levi. 2006. Language Support for Fast and Reliable Message-Based Communication in Singularity OS. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys)*. ACM, 177–190. https://doi.org/10.1145/1217935.1217953

[19] Flutter. 2026. Flutter Architectural Overview. https://docs.flutter.dev/resources/architectural-overview. Accessed 30 July 2026.

[20] James Golike, Andy Lewis-Pye, and Ehud Shapiro. 2026. Grassroots Social Contracts: From Legal Text to an App (Full Version). *arXiv:2609.06231* (2026).

[21] Google. 2024. Dart Programming Language. https://dart.dev/

[22] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 185–200. https://doi.org/10.1145/3062341.3062363

[23] Daniel Halpern, Ariel D Procaccia, Ehud Shapiro, and Nimrod Talmon. 2024. Federated Assemblies. *Proc AAAI 2025; arXiv preprint arXiv:2405.19129* (2024). https://arxiv.org/abs/2405.19129

[24] Michael Hirsch, William Silverman, and Ehud Shapiro. 1987. Computation Control and Protection in the Logix System. In *Concurrent Prolog: Collected Papers*, Ehud Shapiro (Ed.). Vol. 2. MIT Press, Cambridge, MA, Chapter 20, 28–45.

[25] Galen C. Hunt and James R. Larus. 2007. Singularity: Rethinking the Software Stack. *ACM SIGOPS Operating Systems Review* 41, 2 (2007), 37–49. https://doi.org/10.1145/1243418.1243424

[26] Neil D Jones. 1996. An introduction to partial evaluation. *ACM Computing Surveys (CSUR)* 28, 3 (1996), 480–503.

[27] Idit Keidar, Andrew Lewis-Pye, and Ehud Shapiro. 2025. Constitutional Consensus. *arXiv preprint arXiv:2505.19216* (2025). https://arxiv.org/abs/2505.19216

[28] Idit Keidar and Ehud Shapiro. 2026. Volition Elicitation: Operational Semantics for People and Their Machines. arXiv:2607.14138. arXiv:2607.14138 [cs.DC] https://arxiv.org/abs/2607.14138

[29] Sadegh Keshavarzi, Gregory Chockler, and Alexey Gotsman. 2025. TEE Is Not a Healer: Rollback-Resistant Reliable Storage. In *Proceedings of the 39th International Symposium on Distributed Computing (DISC 2025) (LIPIcs, Vol. 356)*. 39:1–39:18. https://doi.org/10.4230/LIPIcs.DISC.2025.39

[30] Nancy S. Kim. 2013. *Wrap Contracts: Foundations and Ramifications*. Oxford University Press, New York.

[31] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT protocol: Usable decentralized social media. *arXiv preprint arXiv:2402.03239* (2024). https://arxiv.org/abs/2402.03239

[32] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401.

[33] Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. 2023. Grassroots Flash: A Payment System for Grassroots Cryptocurrencies. *arXiv preprint arXiv:2309.13191* (2023). https://arxiv.org/abs/2309.13191

[34] Andy Lewis-Pye and Ehud Shapiro. 2026. Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines. *arXiv preprint arXiv:2604.25596* (2026). https://arxiv.org/abs/2604.25596

[35] Essam Mansour, Andrei Vlad Sambra, Sandro Hawke, Maged Zereba, Sarven Capadisli, Abdurrahman Ghanem, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. A Demonstration of the Solid Platform for Social Web Applications. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW)*. ACM, 223–226. https://doi.org/10.1145/2872518.2890529

[36] Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. 2017. ROTE: Rollback Protection for Trusted Execution. In *Proceedings of the 26th USENIX Security Symposium*. https://www.usenix.org/conference/

usenixsecurity17/technical-sessions/presentation/matetic

[37] Matrix.org Foundation. 2019. *Matrix Specification.* Technical Specification. The Matrix.org Foundation. https://spec.matrix.org

[38] Colin Mierowsky, Stephen Taylor, Ehud Shapiro, Jacob Levy, and Shmuel Safra. 1985. *The Design and Implementation of Flat Concurrent Prolog.* Technical Report CS85-09. Department of Computer Science, The Weizmann Institute of Science, Rehovot, Israel.

[39] Mark S Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* Ph. D. Dissertation. Johns Hopkins University.

[40] Nostr Protocol. 2024. NIP-01: Basic Protocol Flow Description. https://github.com/nostr-protocol/nips/blob/master/01.md. Accessed 2026.

[41] Evan Prodromou, Christopher Lemmer Webber, Jessica Tallon, Amy Guy, and Mark Miller. 2018. ActivityPub. https://www.w3.org/TR/activitypub/. https://www.w3.org/TR/activitypub/ W3C Recommendation.

[42] Pierre-Joseph Proudhon and John Beverley Robinson. 2004. *General idea of the revolution in the nineteenth century.* Courier Corporation. Original work published 1851.

[43] Margaret Jane Radin. 2013. *Boilerplate: The Fine Print, Vanishing Rights, and the Rule of Law.* Princeton University Press, Princeton, NJ.

[44] Aravindh Raman, Sagar Joglekar, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. 2019. Challenges in the decentralised web: The mastodon case. In *Proceedings of the internet measurement conference*. 217–229. https://doi.org/10.1145/3355369.3355572

[45] Eugen Rochko. 2016–. Mastodon: A Decentralized Social Network Based on Open Web Protocols. https://joinmastodon.org. https://joinmastodon.org/

[46] Shmuel Safra and Ehud Shapiro. 1987. Meta-interpreters for real. In *Concurrent Prolog: Collected Papers*. Vol. 2. MIT Press, Cambridge, MA, Chapter 25, 166–179. Also in Information Processing 86, pp. 271–278, North-Holland, 1986.

[47] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. https://doi.org/10.1145/359168.359176

[48] Ehud Shapiro. 1989. The family of concurrent logic programming languages. *ACM Computing Surveys (CSUR)* 21, 3 (1989), 413–510. https://doi.org/10.1145/72551.72555

[49] Ehud Shapiro. 2022. Grassroots Currencies: Turning Mutual Trust into Liquidity. *arXiv preprint arXiv:2202.05619* (2022). https://arxiv.org/abs/2202.05619

[50] Ehud Shapiro. 2023. Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*. LIPICS, Italy, 47:1–47:7. https://arxiv.org/abs/2301.04391

[51] Ehud Shapiro. 2023. Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp. In *OASIS '23* (Rome, Italy). Association for Computing Machinery. https://doi.org/10.1145/3599696.3612898

[52] Ehud Shapiro. 2024. A Grassroots Architecture to Supplant Global Digital Platforms by a Global Digital Democracy. *arXiv:2404.13468, Proceedings of DAWO'24* (2024). https://arxiv.org/abs/2404.13468

[53] Ehud Shapiro. 2025. Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots. *arXiv preprint arXiv:2511.03286* (2025). https://arxiv.org/abs/2511.03286

[54] Ehud Shapiro. 2025. GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133* (2025). https://arxiv.org/abs/2510.15747

[55] Ehud Shapiro. 2026. Child-Safe Social Networking: From a Grassroots Social Contract to an App. *In preparation* (2026).

[56] Ehud Shapiro. 2026. Corralling AI Risks with Grassroots Social Contracts. *In preparation* (2026).

[57] Ehud Shapiro. 2026. Grassroots Bonds: Financing by the People, for the People. arXiv:2603.13671. arXiv:2603.13671 https://arxiv.org/abs/2603.13671

[58] Ehud Shapiro. 2026. Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations. *Theoretical Computer Science* (2026), 116306. https://doi.org/doi.org/10.1016/j.tcs.2026.116306 Also 27th ICDCN, arXiv:2502.11299.

[59] Ehud Shapiro. 2026. Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version). *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26* (2026). https://arxiv.org/abs/2602.06934

[60] Ehud Shapiro. 2026. Types for Grassroots Logic Programs (Full version). *arXiv preprint arXiv:2601.17957* (2026). https://arxiv.org/abs/2601.17957

[61] William Silverman, Michael Hirsch, Avshalom Houri, and Ehud Shapiro. 1988. The Logix system user manual Version 1.21. In *Concurrent Prolog: Collected Papers*. MIT Press, 46–77.

[62] Raoul Strackx and Frank Piessens. 2016. Ariadne: A Minimal Approach to State Continuity. In *25th USENIX Security Symposium (USENIX Security)*. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/strackx

[63] Nimrod Talmon and Ehud Shapiro. 2025. Grassroots Federation: Fair Democratic Governance at Scale. *arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26* (2025). https://arxiv.org/abs/2505.02208

[64] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM conference on information-centric networking*. 1–11. https://doi.org/10.1145/3357150.3357396

[65] Kazunori Ueda. 1985. *Guarded Horn Clauses.* Technical Report TR-103. ICOT.

[66] W3C. Retrieved 2022 https://www.w3.org/TR/did-core/. Decentralized Identifiers (DIDs) v1.0. https://www.w3.org/TR/did-core/

[67] Weixin. 2026. Weixin Mini Program: Host Environment. https://developers.weixin.qq.com/miniprogram/en/dev/framework/quickstart/framework.html. Weixin Mini Program developer documentation; accessed 4 September 2026.

[68] Gavin Wood. 2014. Ethereum: A secure decentralised generalised transaction ledger.

[69] Yuqing Yang, Chao Wang, Yue Zhang, and Zhiqiang Lin. 2023. SoK: Decoding the Super App Enigma: The Security Mechanisms, Threats, and Trade-offs in OS-alike Apps. *arXiv preprint arXiv:2306.07495* (2023). https://arxiv.org/abs/2306.07495

[70] Yuqing Yang, Yue Zhang, and Zhiqiang Lin. 2022. Cross Miniapp Request Forgery: Root Causes, Attacks, and Vulnerability Detection. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 3079–3092. https://doi.org/10.1145/3548606.3560597

[71] Yue Zhang, Yuqing Yang, and Zhiqiang Lin. 2023. Don't Leak Your Keys: Understanding, Measuring, and Exploiting the AppSecret Leaks in Mini-Programs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2411–2425. https://doi.org/10.1145/3576915.3616591

[72] Philip Zimmermann. 1994. PGP User's Guide, Volume I: Essential Topics. PGP version 2.6.2, revised 11 October 1994; published as The Official PGP User's Guide, MIT Press, 1995.

[73] Philip Zimmermann and Jon Callas. 2009. The Evolution of PGP's Web of Trust. In *Beautiful Security*. O'Reilly Media, Chapter 7, 107–130.

[74] Shoshana Zuboff. 2019. *The age of surveillance capitalism: The fight for a human future at the new frontier of power.* Public Affairs, US. https://www.hachettebookgroup.com/titles/shoshana-zuboff/the-age-of-surveillance-capitalism/9781610395694/

Use of AI

Generative AI (Claude, by Anthropic) was used to derive the implementation from the specification this paper presents, and in drafting and revising the text.

A The Operating System

The boxed contracts GPE, GSG, GPD and GAS, with their pragmas, say what the persons of the Grassroots App may do and with whose consent, and what their program shows them (Sections 2, 4 and 7). This appendix says what the Grassroots App does around them: the program that runs the four contracts, and what it does by hand, in the order in which it happens. The contracts and this appendix together specify the Grassroots App; the code is `programs/social/graph/core` of the repository.

The program. The compiler of the cascade prints one platform from the four contracts, GPE, GSG, GPD and GAS, with the persons' roles of GRV and GDS, the rendezvous server's and the directory's (GNA, in preparation), as it prints one from any contract that extends others [20, 28]; compiled to GLP it is the procedure `agent(Peer, NetIn, NetOut, Thread, Asks)` and its dispatcher: the agent reads the messages `msg(Q, C)` of its peers on `NetIn`, writes its own on `NetOut`, writes its view and its notices on `Thread`, and asks its person its request questions and its approval questions on `Asks`, which the dispatcher draws on the person channel [28]. The platform knows persons by their keys and conversations by nothing: a channel, a program as a value and a signature never pass through it. The *operating system* is the GLP program written by hand around the platform, which gives it its person and its network and does what the clauses below need done; it is to the Grassroots App's platform what the entry `miniapp/3` of an app's directory, the same file in every app, is to the app's platform (Section 2).

Boot. The runtime posts to the Grassroots App the initial goal of every agent, `agent(ch(_?, _), ch(_?, _))`, the person channel and the network channel, and holds their other ends [54, 59]; the person's first act names the entry, with the person's key, the keys of the persons the person knows, none at a first start, and the services the app is configured with, its rendezvous servers and its directory, each by key and address [59]; the configuration is a file of the repository the app reads when it is built, written by the deployment whenever a service's key or address changes. The networking layer starts empty at every launch of the app; the runtime keeps every standing declaration in its store and makes each again to the new layer at a restart, before any goal runs, so the operating system declares nothing again [59]. Where the runtime refuses an update of the app at restart, its program's types having changed, and runs the program it kept [59], the operating system tells its person so; the platform's view shows the version that runs. The operating system then runs the platform's agent with the person's key as its `Peer`, its dispatcher on the person channel, and the procedures below; under the boot harness one Grassroots App runs per isolate, each with its own key pair, as on a smartphone.

The person. The person sees what the platform draws, its questions and its view with the tabs the pragmas name, and nothing else: the operating system draws no screen of its own, and tells the person nothing but through the platform's notices.

Nearby. At start the operating system declares its rendezvous servers, the trust of `pan` and of `lan` closed and, for each of the two, `nearby(Underlay, Events)` to the layer, and `reachable(Events)`, the peers connected and lost over the WAN, and reads the streams from then on, each agent nearby as it is seen and as it goes, by its key; on GPE:1 it declares the two trusts open, and closed again when the place is closed. A key seen under open trust that the person does not know is a phone whose person opened the place too: the operating system with the smaller key puts GPE:2 to the platform, which takes it on each person's standing request, made by Open place, so the two know each other. The operating system declares to the layer, as a known peer, every person its person knows, so that the layer recognises them wherever they are; the persons nearby the person knows are in the platform's view, and the person befriends one by picking them there (Section 4.2).

Messages. A message the platform sends to a friend goes over the friend channel (below); a message to a person who is not a friend — the request of acquaintance, GKK:2, and an introduction's, GKK:3 — goes as a cold call, `msg(Q, C)` on the network output stream by `send_to_net/1`, the one way to reach an agent with which no variable is shared (Section 3) [54]. A message arriving on the network input stream or on a friend channel joins the agent's `NetIn`. The platform's messages are the start, the votes and the outcome of each of its transactions [28].

The channel to a person one knows. GKK:2 or GKK:3 commits at both parties, and the two know each other. The party with the smaller key makes a channel by `new_channel/2` and sends one end to the other, as a cold call where the two share no variable and over the introducer's channels at an introduction; the networking layer has attested each to the other on the session beneath (Section 8.1, GNA), and the operating system adopts the channel only where the layer reports the peer attested. On the channel each first tells the other the earliest version it is compatible with, reads the other's version from the layer's attestation by `peer_app/2` [54], and the two go on only if the two ranges intersect (Section 8.1). The channel carries the platform's messages to that person, the handshakes of programs and the root channel ends of conversations between participants who know each other. On GSG:1 the two are friends, and the channel is their *friend channel*: each signs `attest(PkA, PkB)` with `sign/3`, `PkA` the

smaller key, and gives the signature to the other, for reconnection (Section 4.2), and the channel carries from then on the operating system's offers of programs, requests for programs, programs and invitations. On GSG:2 the party that ends the friendship sends `unfriend` on the channel and both close it; the programs the two shared keep no conversation between them (below, Leaving).

Programs. A program is a value: compiled code with its compiler's certificate, answering `contract(Text, Schemas)` and `version(Current, Back)` (Sections 5 and 3). The platform's state holds, by GPD, the programs the person installed, the developers the person trusts and the programs offered to the person, each program by its source identity; the operating system holds, for each program it has, the value, and for each installed program its two channels and whether it runs. On GPD:1 the developer's operating system signs the program's identities with `sign/3`. On GPD:3 Carol's operating system sends Alice's the program over the friend channel, with Carol's signature added; Alice's checks it by `signature/2` and `decompose_module/4`: its developer, the first signer, is one Alice trusts, the schemas pass the checker, and the compiler's certificate matches the code (GPD's implementation pragmas), and holds it as offered. On GPD:4 the operating system records the program installed, with no execution yet.

Invitation. An installed program asks for an invitation when it first sends a message to a friend it has no conversation with: it writes `invite(Bob)` up its social-graph channel and holds the message (Section 5). Alice's operating system puts GPD:5's request to the platform in Alice's name, she having willed the act inside the program, and the platform starts the transaction with Bob, its start reaching Bob's platform over the friend channel. Bob's operating system looks the program's source identity up; where Bob lacks the program it asks Alice's operating system for it over the friend channel, Alice's sends it with her signature added, and Bob's checks it as above; only then does the platform ask Bob, who reads the contract, and Bob answers. Where he declines, a program fetched for the invitation is dropped, and the operating system tells the program `declined(Bob)` down its social-graph channel, as it does where Bob lacks the program and is not a friend, or is unknown; the program drops what it held for Bob. Two invitations that cross open one conversation: the one from the smaller key stands. Where GPD:5 commits, Alice's operating system makes the root channel by `new_channel/2`, keeps one end and sends the other to Bob's over the friend channel, and the two run the handshake of Section 8.1 over the friend channel: the program's version and earliest compatible version, the type identity of the root channel, and, where the two have not been in contact through the program, their undertakings, signed with `sign/3`; on disagreement the root channel is closed and nothing is delivered.

Activation. Each side then hands its execution of the program its end. Where the program is not running, the operating system activates it: `opaque_type/3` gives the two types at which the program declares its entry, `find_type/3` the type identity of the entry template's instance at them, and `run/3` posts the entry goal, with the person's key, a fresh person channel, and the social-graph channel, under that type identity (Section 3) [60]; the dispatcher draws the program's person channel beside the Grassroots App's own. Where it runs, the end goes to the running execution. The end goes down the social-graph channel as `conversation(Key, End)`, the friend's key and the root channel's end; what the program sends on the end from then on is its own, of a type the operating system holds as `Opaque` and never inspects. A program with no conversation, installed from a store or activated alone, is activated the same way on the person's act, with no end to hand.

Whom the Grassroots App knows. The operating system holds a channel to each agent its person knows, and no other. An end of a variable that reaches it from an agent it does not know is held by the runtime and reported to the operating system [59]; the operating system asks its person whether to know that agent, by its key, and on yes makes the channel to it as above, attested, and releases the end by `authorise_link(Name, authorise)`; on no, `authorise_link(Name, refuse)`. A program's conversation with a participant its person knows but has no conversation with opens on the program's first message to them, as at an invitation, over the channel to that agent (Section 4.2).

Leaving. On GPD:6 the operating system closes every conversation of the program at the person who leaves, whose peers see a crash, and ends the execution; on GSG:2 it closes the conversations the two friends had in every program. When the stream of the person's acts closes, the Grassroots App ends: the operating system closes its friend channels and every program's conversations (Section 4.1).

What the operating system calls. The operating system is the one program of the Grassroots App that calls the OS-privileged predicates of Section 2: `send_to_net/1` and `send_to_user/1`, the five seam predicates of the networking layer, and `authorise_link/2`; beside them it calls `sign/3`, `signature/2`, `decompose_module/4`, `opaque_type/3`, `find_type/3` and `run/3`, which a program may call too [54]. The reconnection of a friend channel through a rendezvous agent, and the invitation link, are the operating system's as Section 4.2 and Section 4.2 state them, over the seam predicates (GNA, in preparation). The budget and the control stream of a program (Section 8.3) and the recovery of Section 4.3 are not yet the operating system's (Section 9).

B The Grassroots App and the Super-Apps

Section 10 states the architectural difference, the locus of control. This appendix presents the detail, over three questions: what the host is and what it grants, which security mechanisms each design has, and what one mini-app can do to another.

Table 1 of Section 1 answers the first, and three rows are left to add here. *Execution*: a mini-app's script runs on a JavaScript engine thread and its markup on WebView threads, the client passing between the two [67], where here goals are posted to a certified program by `run/3` in the person's one heap [54, 59], and, under the design of Section 8.3, under a meta-interpreter that bounds them. *Withdrawal*: the operator can remove a mini-app from every device, where here nobody can remove a program from a person's smartphone. *Essential agents*: the operator, so the platform is centralised [53], where here there are none.

What separates two mini-apps. A super-app separates mini-apps by giving each its own storage and by filtering the channel between them [69]. Here the separation is reachability, stated in Section 8.2.

C The Meta-Interpreter's Streams

The meta-interpreter of Section 8.3 joins the request streams of two conjuncts into one with `merge/3`, and gives each conjunct the control stream with `distribute/3`; the messages of the control stream are constants, which is what lets one reader of it be written twice in a clause [54]:

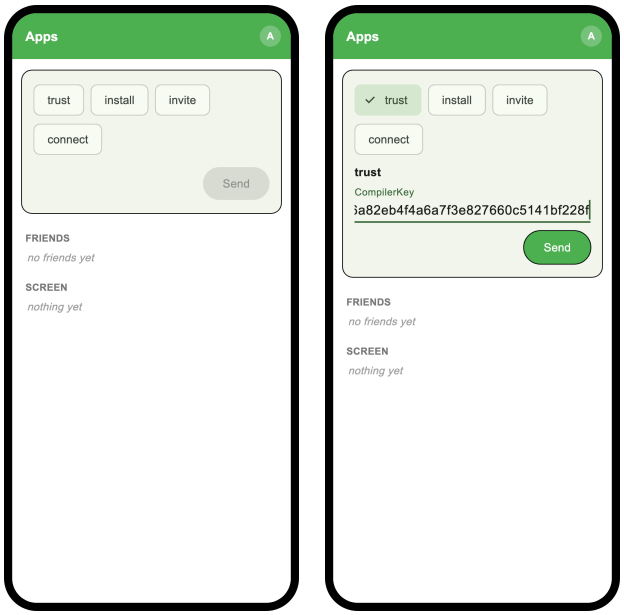
```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge([], Ys, Ys?).
merge(Xs, [], Xs?).

distribute([X|Xs], [X?|Ys?], [X?|Zs?]) :-
    constant(X?) | distribute(Xs?, Ys, Zs).
distribute([], [], []).
```

When either request stream closes, the merged stream continues as the other, so a chain of merges left by a recursive goal shortens as its conjuncts terminate. The Grassroots App reads the merged stream of each program and assigns each `more(N)` it carries; it writes `abort` on the control stream to end the program, and closes the stream otherwise. The merges, the distributes and the Grassroots App's reading are the meta-interpreter's own reductions and are not charged to the program; each merge reduction is caused by one request, and a request is made only by a goal whose budget was spent, so the meta-interpreter's reductions are bounded by the program's.

D The Prototype's Screens

Every element of every screen below is the default widget of the type of a question the Grassroots App's own program asks its person, and none is written by hand [28]. All are rendered by the user interface in its test harness, at the size of a phone's screen.



(a) The four acts (b) Trust a compiler

(c) Install a program (d) After Connect

Figure 4. The Grassroots App's own person interface, as Alice sees it. (a) The Apps panel: the menu of the four acts a person wills of their Grassroots App — trust a compiler, install a program, invite a friend, connect — and her friends and screen, both empty. (b) The Trust form, a compiler's key typed into its one field, `CompilerKey`. (c) The Install form, the name the program takes, `currency`, and the certified program, `currency.glpw`. (d) After Connect: Bob is a friend.

Mechanism [69]	Here
S1 Permission mechanism	The Grassroots App grants a program two GLP channels, to the social graph and to the user interface, and nothing else (Section 2)
S2 Sandboxing: a storage space per mini-app	The Grassroots App gives each program private storage, sealed by the platform (Section 2)
S3 API restriction	The Grassroots App grants two GLP channels; there is no further set to restrict
S4 Cross-miniapp allowlisting	None. A program communicates with executions of itself and with nothing else (Section 8)
S5 Designated distribution channel, hot update forbidden	Distribution by a developer or a store to the people who know them, and friend to friend. The program carries its developer's and its compiler's signatures and the contract it is compiled from, which the receiving Grassroots App checks (Section 5)
S6 Domain allowlisting	None. A program reaches other people along edges of the social graph and fetches no page
S7 Secure communication	Noise sessions in the networking layer, and the attestation and the handshake above it (Section 8.1)
S8 Role-based access control for developers	None. There are no developer accounts and no operator cloud
S9 Data encryption under a master key	None. No key is held by an operator
S10 Token-based access to services	None. There are no operator services
S11 User token isolation	None. A person's identity is their own key
S12 Code vetting	None. The developer's attestation, the compiler's certificate, the checker run on the program's contract, the person's trust in the developer and the type check at activation take its place (Section 8.4)
S13 Account protection	None. There are no accounts

Table 3. The thirteen security mechanisms of the super-app, and what each becomes here. Six are built differently, six presuppose an operator and have no counterpart, and code vetting is replaced by certification and a check the machine performs.

Threat [69]	Here
T1 Data leakage through a flawed permission	Does not arise. What the Grassroots App grants is a GLP channel, and the phone's own permissions belong to the Grassroots App
T2 Data leakage through cross-platform discrepancies	Confined to the virtual machine. The social graph and every program are GLP programs on both phone operating systems, and what differs beneath them is Dart [59]
T3 Data leakage through hidden interfaces	Does not arise. The catalogue of guards, system predicates and body kernels is complete [54]; an undocumented kernel is a defect of the implementation
T4 Injection over the cross-miniapp channel [70]	Does not arise. There is no cross-miniapp channel (Section 8.2), and the runtime puts the signing program's source identity into every signature, so one program cannot present itself as another (Section 8.1)
T5 Vetting bypassed by a hot update	Does not arise. There is no vetting, and the certificate binds the code the peer runs
T6 Vetting bypassed by identity confusion	Does not arise. There is no vetting
T7 Data leakage through misuse of the master key [71]	Does not arise. There is no such key
T8 Data leakage through an abused service token	Does not arise. There is no such token
T9 Weak token isolation, letting two mini-apps of one developer collude	Does not arise. Two programs one person installed are goals in one heap, and hold no variable in common (Section 8.2)
T10 Vetting bypassed by evasive malware	Bounded. Separation bounds whom a program may reach, the meta-interpreter bounds what it consumes and lets the person abort it; what it tells the person who installed it hinges on its developer (Section 8.4)

Table 4. The ten threats of the super-app. Eight do not arise, one is confined to the virtual machine, and one is bounded